Multi threaded programming and Lambda expressions

1) Thread Priorities

Problem Statement:

Write a Java program to create multiple threads and demonstrate different thread priorities using MIN_PRIORITY, NORM_PRIORITY, and MAX_PRIORITY.

Code:

```
class PriorityThread extends Thread {
    public PriorityThread(String name) {
        super(name);
    }

    public void run() {
        System.out.println(getName() + " is running with priority " + getPriority());
    }
}

public class ThreadPriorityDemo {
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread("Low Priority Thread");
        PriorityThread t2 = new PriorityThread("Normal Priority Thread");
        PriorityThread t3 = new PriorityThread("High Priority Thread");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
        t3.start();
    }
}
```

```
C:\Users\trivi\Javalab>javac ThreadPriorityDemo.java

C:\Users\trivi\Javalab>java ThreadPriorityDemo.java
High Priority Thread is running with priority 10
Low Priority Thread is running with priority 1
Normal Priority Thread is running with priority 5

C:\Users\trivi\Javalab>
```

2) Execution Order of Threads

Problem Statement:

Write a Java program to create three threads with different priorities and observe their execution order.

Code:

```
class OrderThread extends Thread {
    public OrderThread(String name) {
        super(name);
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " - step " + i +
```

```
                " - priority " + getPriority());
        }
    }
}

public class ThreadOrderDemo {
    public static void main(String[] args) throws InterruptedException {
        OrderThread low = new OrderThread("Low");
        OrderThread normal = new OrderThread("Normal");
        OrderThread high = new OrderThread("High");

        low.setPriority(Thread.MIN_PRIORITY);
        normal.setPriority(Thread.NORM_PRIORITY);
        high.setPriority(Thread.MAX_PRIORITY);

        low.start();
        normal.start();
        high.start();

        low.join();
        normal.join();
        high.join();
        System.out.println("All threads completed.");
    }
}
```

```
C:\Users\trivi\Javalab>javac ThreadOrderDemo.java

C:\Users\trivi\Javalab>java ThreadOrderDemo.java
Low - step 1 - priority 1
High - step 1 - priority 10
High - step 2 - priority 10
High - step 3 - priority 10
High - step 4 - priority 10
High - step 5 - priority 10
Normal - step 1 - priority 5
Low - step 2 - priority 1
Normal - step 2 - priority 5
Low - step 3 - priority 1
Normal - step 3 - priority 5
Low - step 4 - priority 1
Normal - step 4 - priority 5
Low - step 5 - priority 1
Normal - step 5 - priority 5
All threads completed.

C:\Users\trivi\Javalab>
```

3) Synchronized Method

Problem Statement:

Write a Java program to demonstrate synchronization using a synchronized method for accessing a shared counter.

Code:

```
class SharedCounter {
    private int count = 0;

    public synchronized void increment() {
        count++;
```

```
        System.out.println(Thread.currentThread().getName()
                + " incremented counter to " + count);
    }

    public int getCount() {
        return count;
    }
}

public class SynchronizedMethodDemo {
    public static void main(String[] args) throws InterruptedException {
        SharedCounter counter = new SharedCounter();

        Runnable task = () -> {
            for (int i = 1; i <= 5; i++) {
                counter.increment();
            }
        };

        Thread t1 = new Thread(task, "Thread-1");
        Thread t2 = new Thread(task, "Thread-2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final counter value: " + counter.getCount());
    }
}
```

```
C:\Users\trivi\Javalab>javac SynchronizedMethodDemo.java

C:\Users\trivi\Javalab>java SynchronizedMethodDemo.java
Thread-1 incremented counter to 1
Thread-1 incremented counter to 2
Thread-1 incremented counter to 3
Thread-1 incremented counter to 4
Thread-1 incremented counter to 5
Thread-2 incremented counter to 6
Thread-2 incremented counter to 7
Thread-2 incremented counter to 8
Thread-2 incremented counter to 9
Thread-2 incremented counter to 10
Final counter value: 10

C:\Users\trivi\Javalab>
```

4) Synchronized Block

Problem Statement:

Write a Java program to demonstrate synchronization using a synchronized block.

Code:

```
class BlockCounter {
    private int count = 0;

    public void increment() {
        synchronized (this) {
            count++;
            System.out.println(Thread.currentThread().getName()
                    + " incremented counter to " + count);
```

```
        }
    }

    public int getCount() {
        return count;
    }
}

public class SynchronizedBlockDemo {
    public static void main(String[] args) throws InterruptedException {
        BlockCounter counter = new BlockCounter();
        Runnable task = () -> {
            for (int i = 1; i <= 5; i++) {
                counter.increment();
            }
        };

        Thread t1 = new Thread(task, "Thread-1");
        Thread t2 = new Thread(task, "Thread-2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final counter value: " + counter.getCount());
    }
}
```

```
C:\Users\trivi\Javalab>javac SynchronizedBlockDemo.java

C:\Users\trivi\Javalab>java SynchronizedBlockDemo.java
Thread-1 incremented counter to 1
Thread-1 incremented counter to 2
Thread-1 incremented counter to 3
Thread-1 incremented counter to 4
Thread-1 incremented counter to 5
Thread-2 incremented counter to 6
Thread-2 incremented counter to 7
Thread-2 incremented counter to 8
Thread-2 incremented counter to 9
Thread-2 incremented counter to 10
Final counter value: 10

C:\Users\trivi\Javalab>
```

5) Race Condition and Synchronization

Problem Statement:

Write a Java program to demonstrate a race condition and eliminate it using synchronization.

Code:

```
class SafeCounter {
    private int value = 0;

    public synchronized void increment() {
        value++;
    }

    public int getValue() {
        return value;
```

```
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {
        SafeCounter counter = new SafeCounter();
        Runnable task = () -> {
            for (int i = 1; i <= 1000; i++) {
                counter.increment();
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Expected value: 2000");
        System.out.println("Actual value: " + counter.getValue());
        System.out.println("Synchronization prevents lost updates.");
    }
}
```

```
C:\Users\trivi\Javalab>javac RaceConditionDemo.java

C:\Users\trivi\Javalab>java RaceConditionDemo.java
Expected value: 2000
Actual value: 2000
Synchronization prevents lost updates.

C:\Users\trivi\Javalab>
```

6) Synchronized Ticket Booking

Problem Statement:

Write a Java program to implement a synchronized ticket-booking system where multiple threads attempt to book tickets simultaneously.

Code:

```
class TicketBooking {
    private int availableTickets = 5;

    public synchronized void bookTicket(String passenger, int numberOfTickets) {
        if (numberOfTickets <= availableTickets) {
            System.out.println(passenger + " booked " + numberOfTickets + " ticket(s).");
            availableTickets -= numberOfTickets;
            System.out.println("Tickets remaining: " + availableTickets);
        } else {
            System.out.println(passenger + " could not book " + numberOfTickets
                    + " ticket(s). Not enough tickets.");
        }
    }
}

public class TicketBookingDemo {
    public static void main(String[] args) throws InterruptedException {
        TicketBooking booking = new TicketBooking();

        Thread t1 = new Thread(() -> booking.bookTicket("Alice", 2));
        Thread t2 = new Thread(() -> booking.bookTicket("Bob", 2));
```

```
        Thread t3 = new Thread(() -> booking.bookTicket("Charlie", 2));

        t1.start();
        t2.start();
        t3.start();
        t1.join();
        t2.join();
        t3.join();
    }
}
```

```
C:\Users\trivi\Javalab>javac TicketBookingDemo.java

C:\Users\trivi\Javalab>java TicketBookingDemo.java
Alice booked 2 ticket(s).
Tickets remaining: 3
Charlie booked 2 ticket(s).
Tickets remaining: 1
Bob could not book 2 ticket(s). Not enough tickets.

C:\Users\trivi\Javalab>
```

7) Functional Interface for Arithmetic

Problem Statement:

Write a Java program to introduce lambda expressions by implementing a functional interface for addition, subtraction, multiplication, and division.

Code:

```
@FunctionalInterface
interface ArithmeticOperation {
    double calculate(double a, double b);
}

public class ArithmeticLambdaDemo {
    public static void main(String[] args) {
        ArithmeticOperation addition = (a, b) -> a + b;
        ArithmeticOperation subtraction = (a, b) -> a - b;
        ArithmeticOperation multiplication = (a, b) -> a * b;
        ArithmeticOperation division = (a, b) -> a / b;

        double x = 20;
        double y = 5;

        System.out.println("Addition: " + addition.calculate(x, y));
        System.out.println("Subtraction: " + subtraction.calculate(x, y));
        System.out.println("Multiplication: " + multiplication.calculate(x, y));
        System.out.println("Division: " + division.calculate(x, y));
    }
}
```

```
C:\Users\trivi\Javalab>javac ArithmeticLambdaDemo.java

C:\Users\trivi\Javalab>java ArithmeticLambdaDemo.java
Addition: 25.0
Subtraction: 15.0
Multiplication: 100.0
Division: 4.0

C:\Users\trivi\Javalab>
```

8) Single and Multiple Parameters

Problem Statement:

Write a Java program to demonstrate a lambda expression with a single parameter and multiple parameters.

Code:

```
@FunctionalInterface
interface SingleParameter {
    int square(int number);
}

@FunctionalInterface
interface MultipleParameters {
    int add(int a, int b);
}

public class LambdaParametersDemo {
    public static void main(String[] args) {
        SingleParameter square = number -> number * number;
        MultipleParameters addition = (a, b) -> a + b;

        System.out.println("Square of 6: " + square.square(6));
        System.out.println("12 + 8: " + addition.add(12, 8));
    }
}
```

```
C:\Users\trivi\Javalab>javac LambdaParametersDemo.java

C:\Users\trivi\Javalab>java LambdaParametersDemo.java
Square of 6: 36
12 + 8: 20

C:\Users\trivi\Javalab>
```

9) Block Lambda Expression

Problem Statement:

Write a Java program to demonstrate a block lambda expression containing multiple statements.

Code:

```
@FunctionalInterface
interface MessageProcessor {
    String process(String message);
}

public class BlockLambdaDemo {
    public static void main(String[] args) {
        MessageProcessor processor = message -> {
            String trimmed = message.trim();
            String upperCase = trimmed.toUpperCase();
            return "Processed message: " + upperCase;
        };

        System.out.println(processor.process("  hello lambda  "));
    }
}
```

```
C:\Users\trivi\Javalab>javac BlockLambdaDemo.java

C:\Users\trivi\Javalab>java BlockLambdaDemo.java
Processed message: HELLO LAMBDA

C:\Users\trivi\Javalab>
```

10) Largest of Two Numbers

Problem Statement:

Write a Java program using a block lambda expression to find the largest of two numbers and return the result.

Code:

```
@FunctionalInterface
interface LargestNumber {
    int findLargest(int a, int b);
}

public class LargestNumberLambdaDemo {
    public static void main(String[] args) {
        LargestNumber largest = (a, b) -> {
            if (a > b) {
                return a;
            } else {
                return b;
            }
        };

        System.out.println("Largest number: " + largest.findLargest(45, 72));
    }
}
```

```
C:\Users\trivi\Javalab>javac LargestNumberLambdaDemo.java

C:\Users\trivi\Javalab>java LargestNumberLambdaDemo.java
Largest number: 72

C:\Users\trivi\Javalab>
```

11) Generic Functional Interface

Problem Statement:

Write a Java program to create a generic functional interface and implement it using lambda expressions for different data types.

Code:

```
@FunctionalInterface
interface Display<T> {
    void show(T value);
}

public class GenericLambdaDemo {
    public static void main(String[] args) {
        Display<Integer> integerDisplay = value ->
                System.out.println("Integer value: " + value);
        Display<String> stringDisplay = value ->
                System.out.println("String value: " + value);
        Display<Double> doubleDisplay = value ->
                System.out.println("Double value: " + value);
```

```
        integerDisplay.show(100);
        stringDisplay.show("Java Lambda");
        doubleDisplay.show(25.75);
    }
}
```

```
C:\Users\trivi\Javalab>javac GenericLambdaDemo.java

C:\Users\trivi\Javalab>java GenericLambdaDemo.java
Integer value: 100
String value: Java Lambda
Double value: 25.75

C:\Users\trivi\Javalab>
```

12) Generic Binary Operation

Problem Statement:

Write a Java program to create a generic functional interface that accepts two values and performs an operation using a lambda expression.

Code:

```
@FunctionalInterface
interface BinaryOperation<T> {
    T operate(T first, T second);
}

public class GenericBinaryOperationDemo {
    public static void main(String[] args) {
        BinaryOperation<Integer> addition = (a, b) -> a + b;
        BinaryOperation<String> joining = (a, b) -> a + " " + b;
        BinaryOperation<Double> multiplication = (a, b) -> a * b;

        System.out.println("Integer addition: " + addition.operate(10, 20));
        System.out.println("String joining: " + joining.operate("Hello", "Java"));
        System.out.println("Double multiplication: "
                + multiplication.operate(2.5, 4.0));
    }
}
```

```
C:\Users\trivi\Javalab>javac GenericBinaryOperationDemo.java

C:\Users\trivi\Javalab>java GenericBinaryOperationDemo.java
Integer addition: 30
String joining: Hello Java
Double multiplication: 10.0
```

13) Display and Filter an ArrayList

Problem Statement:

Write a Java program to use lambda expressions to display and filter elements of an ArrayList.

Code:

```
import java.util.ArrayList;
import java.util.Arrays;

public class ArrayListLambdaDemo {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>(
                Arrays.asList(10, 15, 20, 25, 30, 35));

        System.out.println("All numbers:");
        numbers.forEach(number -> System.out.println(number));
```

```
        System.out.println("Numbers greater than 20:");
        numbers.forEach(number -> {
            if (number > 20) {
                System.out.println(number);
            }
        });
    }
}
```

```
C:\Users\trivi\Javalab>javac ArrayListLambdaDemo.java

C:\Users\trivi\Javalab>java ArrayListLambdaDemo.java
All numbers:
10
15
20
25
30
35
Numbers greater than 20:
25
30
35

C:\Users\trivi\Javalab>
```

14) Sorting Lists Using Lambdas

Problem Statement:

Write a Java program to sort a list of integers and strings using lambda expressions.

Code:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class LambdaSortingDemo {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(
                Arrays.asList(45, 12, 78, 3, 29));
        List<String> names = new ArrayList<>(
                Arrays.asList("Ravi", "Anita", "Kiran", "Bhavna"));

        numbers.sort((a, b) -> a.compareTo(b));
        names.sort((a, b) -> a.compareToIgnoreCase(b));

        System.out.println("Sorted integers: " + numbers);
        System.out.println("Sorted strings: " + names);
    }
}
```

```
C:\Users\trivi\Javalab>javac LambdaSortingDemo.java

C:\Users\trivi\Javalab>java LambdaSortingDemo.java
Sorted integers: [3, 12, 29, 45, 78]
Sorted strings: [Anita, Bhavna, Kiran, Ravi]

C:\Users\trivi\Javalab>
```

15) Thread Using Lambda Expression

Problem Statement:

Write a Java program to create and execute a thread using a lambda expression.

Code:

```
public class LambdaThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Lambda thread: " + i);
            }
        };

        Thread thread = new Thread(task, "LambdaThread");
        thread.start();
        thread.join();

        System.out.println("Main thread completed.");
    }
}
```

```
C:\Users\trivi\Javalab>javac LambdaThreadDemo.java

C:\Users\trivi\Javalab>java LambdaThreadDemo.java
Lambda thread: 1
Lambda thread: 2
Lambda thread: 3
Lambda thread: 4
Lambda thread: 5
Main thread completed.

C:\Users\trivi\Javalab>
```